Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського» Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Організація баз даних

Лабораторна робота №4

«Аналітичні SQL-запити (OLAP)»

Виконала:

студентка групи ІО-46

Чубарева Д.Д.

Номер у списку групи: 16

Перевірив:

Русінов В. В.

Київ – 2026

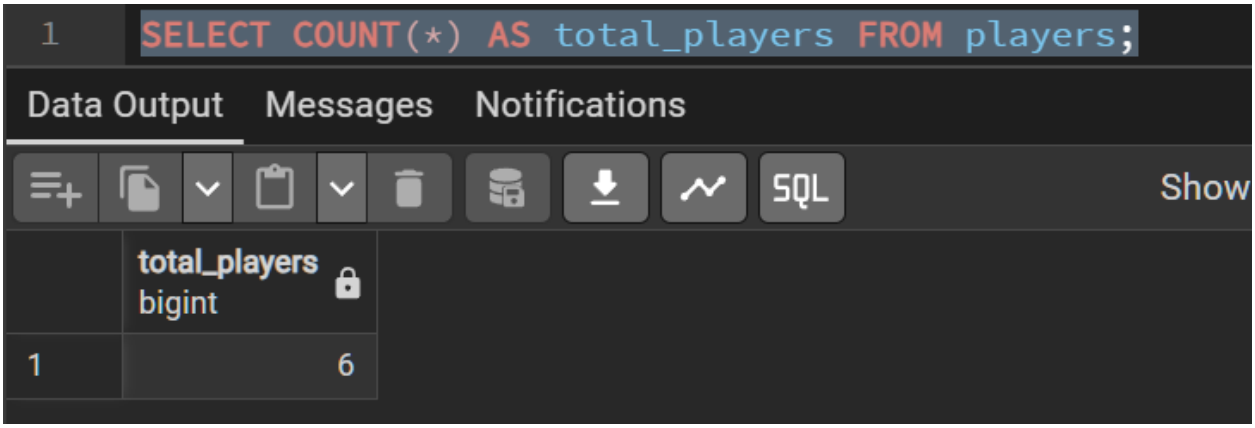
Тема: «Аналітичні SQL-запити (OLAP)».

Цілі:

- Використати агрегатні функції, такі як COUNT, SUM, AVG, MIN та MAX, для обчислення зведеної статистики з ваших даних.
- Написати запити GROUP BY для групування рядків за одним або кількома стовпцями та обчислення агрегатів для кожної групи.
- Використати HAVING для фільтрації результатів згрупованих запитів на основі агрегованих умов.
- Виконувати операції JOIN (принаймні INNER JOIN та LEFT JOIN), щоб об'єднати дані з кількох таблиць.
- Створювати об'єднані запити на агрегацію для кількох таблиць, які об'єднують таблиці та створюють згрупований, агрегований вивід.
- Інтерпретувати результати ваших запитів та поясніть, що робить кожен з них.

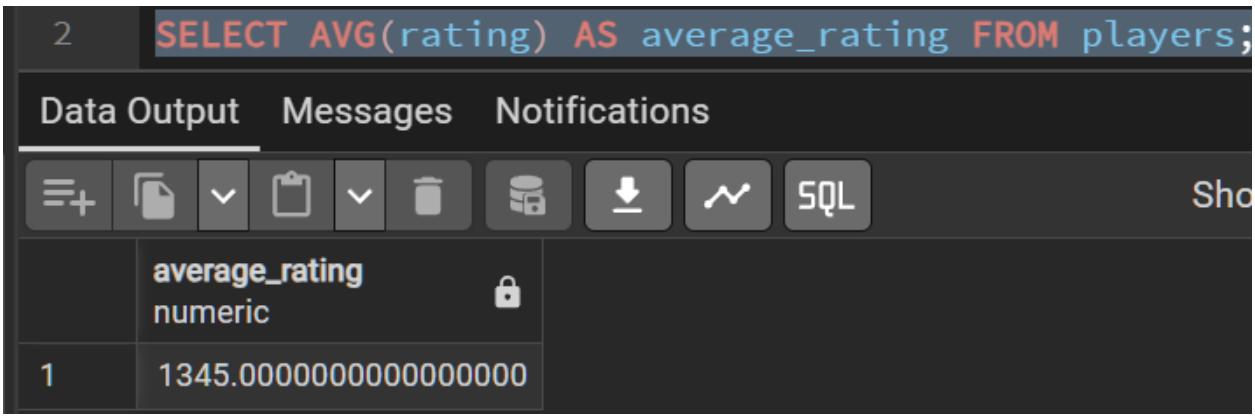
4 запити, що містять агрегаційні функції (SUM, AVG, COUNT, MIN, MAX, GROUP BY)

```
SELECT COUNT(*) AS total_players FROM players;
```



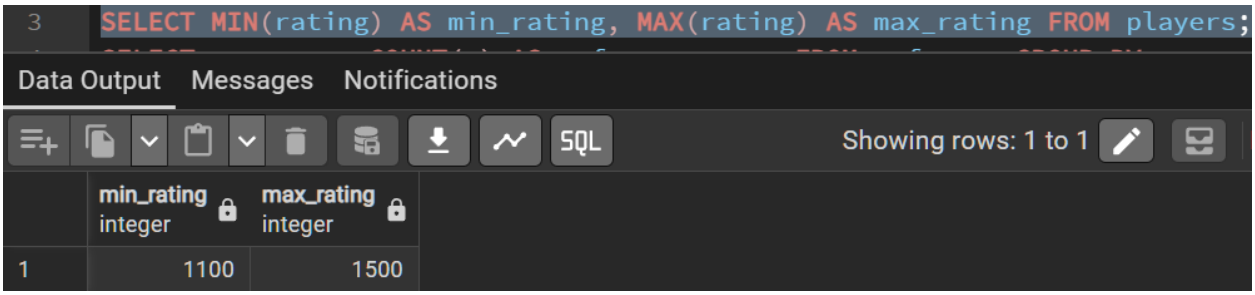
Цей запит рахує загальну кількість гравців, які збережені в базі даних за допомогою функції COUNT.

```
SELECT AVG(rating) AS average_rating FROM players;
```



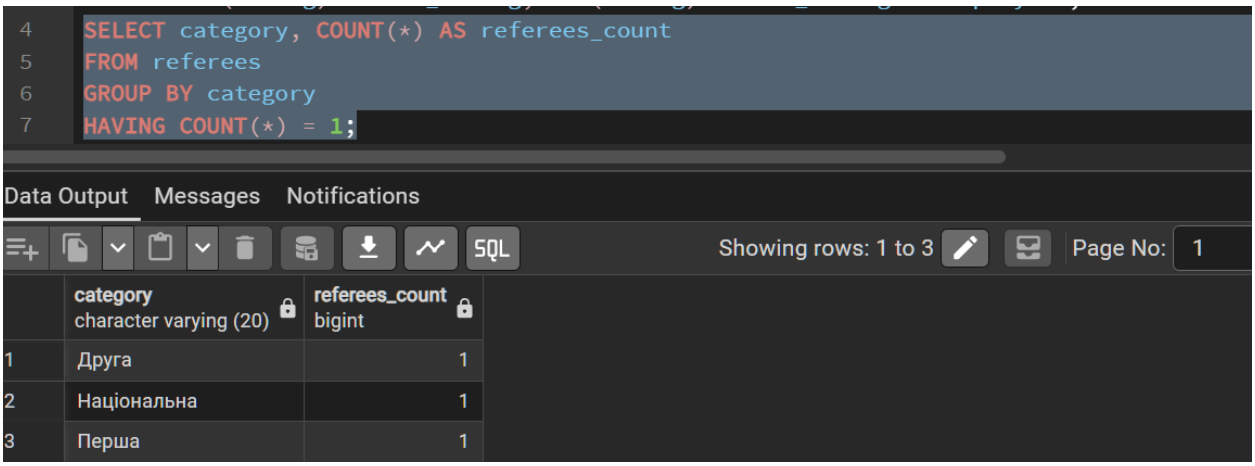
А цей запит обчислює середній рейтинг серед усіх тенісистів у таблиці players за допомогою функції AVG.

```
SELECT MIN(rating) AS min_rating, MAX(rating) AS max_rating FROM players;
```



Тут я знайшла мінімальний та максимальний показники рейтингу серед усіх гравців.

```
SELECT category, COUNT(*) AS referees_count
FROM referees
GROUP BY category
HAVING COUNT(*) = 1;
```



Тут я згрупувала суддів за категоріями і за допомогою HAVING відфільтрувала лише ті категорії, де є рівно один зареєстрований суддя.

3 запити, що використовують різні типи джоінів (INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN)

```
SELECT r.first_name, r.last_name, COUNT(m.match_id) AS total_matches

FROM referees r

INNER JOIN matches m ON r.referee_id = m.referee_id

GROUP BY r.first_name, r.last_name;
```

```
8  SELECT r.first_name, r.last_name, COUNT(m.match_id) AS total_matches
9  FROM referees r
10 INNER JOIN matches m ON r.referee_id = m.referee_id
11 GROUP BY r.first_name, r.last_name;
```

Data Output | Messages | Notifications

Showing rows: 1 to 3 | Page No: 1

	first_name character varying (50)	last_name character varying (50)	total_matches bigint
1	Сюй	Сін	2
2	Наталія	Кудлай	2
3	Ганс-Петер	Верт	1

Тут я об'єднала таблиці суддів та матчів за допомогою INNER JOIN, а потім згрупувала результати за іменем та прізвищем судді. Завдяки агрегатній функції COUNT запит рахує, скільки саме матчів обслуговував кожен суддя в системі. Це приклад багатотабличної агрегації.

```
SELECT p.first_name, p.last_name, tr.payment_status

FROM players p

LEFT JOIN tournament_registrations tr ON p.player_id = tr.player_id;
```

```
8  SELECT p.first_name, p.last_name, tr.payment_status
9  FROM players p
10 LEFT JOIN tournament_registrations tr ON p.player_id = tr.player_id;
```

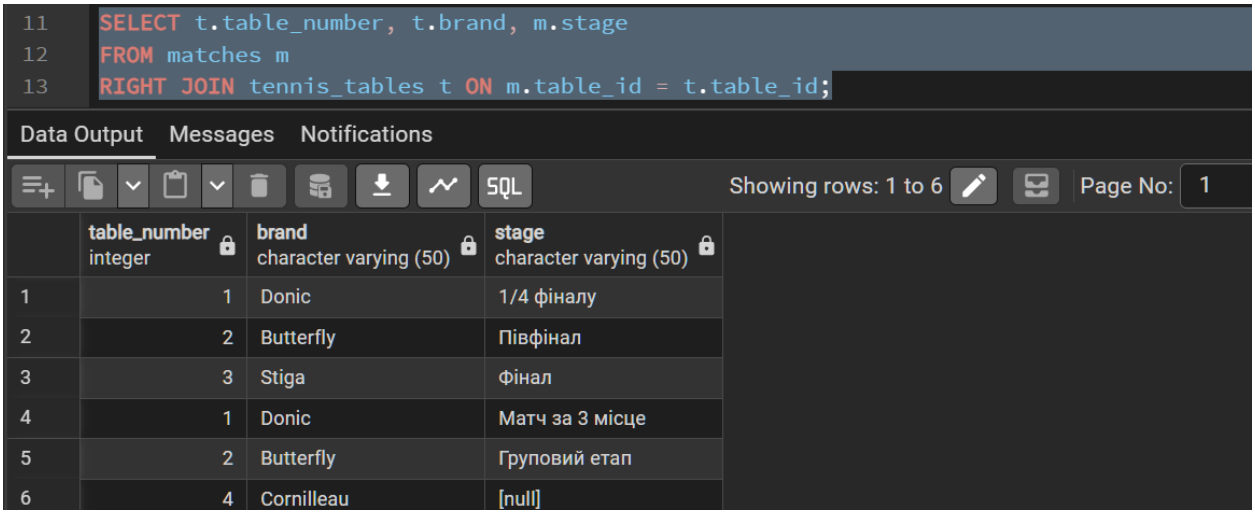
Data Output | Messages | Notifications

Showing rows: 1 to 6 | Page No: 1

	first_name character varying (50)	last_name character varying (50)	payment_status character varying (20)
1	Фан	Чжендон	Оплачено
2	Олександр	Дідух	Оплачено
3	Ван	Чуцін	Оплачено
4	Віктор	Барна	Оплачено
5	Назар	Третяк	Оплачено
6	Карлос	Алькарас	[null]

Він показує імена всіх гравців з бази та їхній статус оплати турніру. Навіть якщо гравець ще нікуди не зареєструвався, він все одно виведеться в результатах, так сталось з Карлосом.

```
SELECT t.table_number, t.brand, m.stage
FROM matches m
RIGHT JOIN tennis_tables t ON m.table_id = t.table_id;
```



	table_number integer	brand character varying (50)	stage character varying (50)
1	1	Donic	1/4 фіналу
2	2	Butterfly	Півфінал
3	3	Stiga	Фінал
4	1	Donic	Матч за 3 місце
5	2	Butterfly	Груповий етап
6	4	Cornilleau	[null]

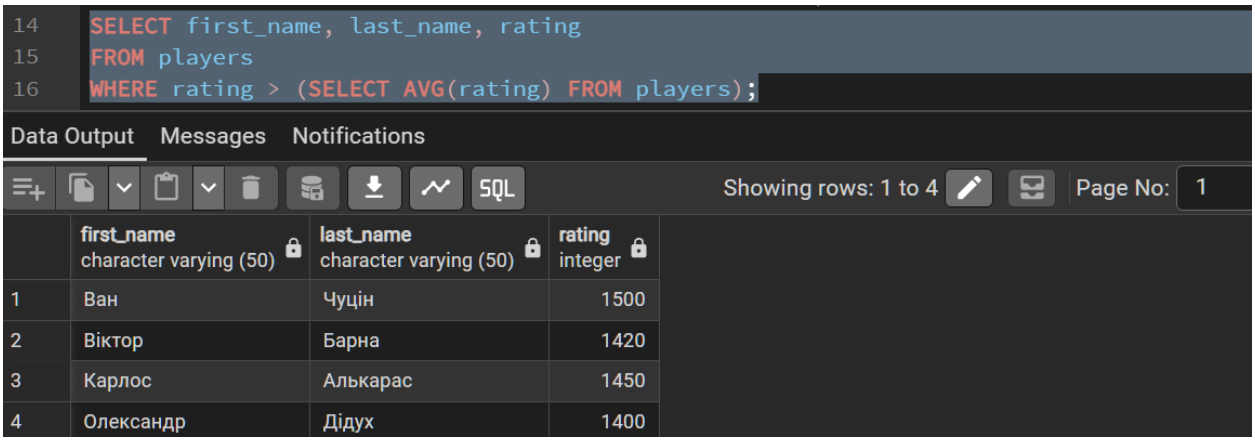
А цей виводить номер столу, його бренд та етап матчу. Завдяки RIGHT JOIN ми точно побачимо в списку всі наявні столи, навіть ті, на яких ще не проводилися ігри.

3 запити з використанням підзапитів (вибірка з підзапитом у SELECT, WHERE, або HAVING)

```
SELECT first_name, last_name, rating

FROM players

WHERE rating > (SELECT AVG(rating) FROM players);
```



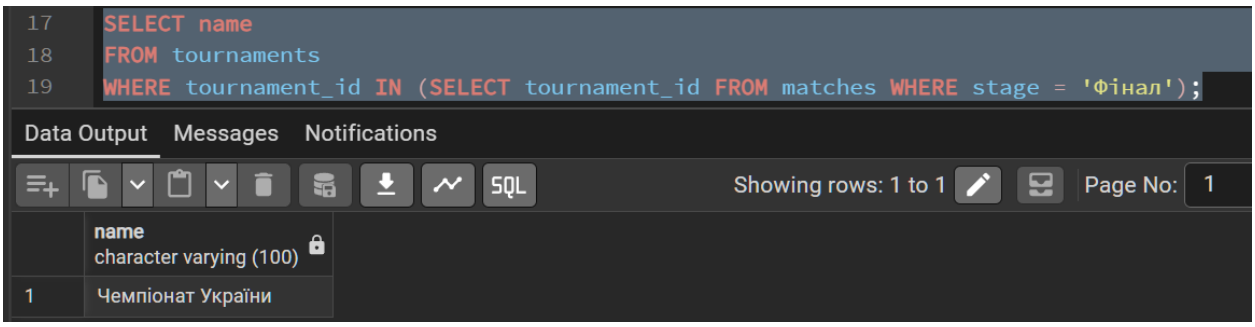
	first_name character varying (50)	last_name character varying (50)	rating integer
1	Ван	Чуцін	1500
2	Віктор	Барна	1420
3	Карлос	Алькарас	1450
4	Олександр	Дідух	1400

Запит виводить імена та рейтинг тих гравців, у яких рейтинг вищий за середній показник по всій базі, за допомогою WHERE.

```
SELECT name

FROM tournaments

WHERE tournament_id IN (SELECT tournament_id FROM matches WHERE stage =
'Фінал');
```

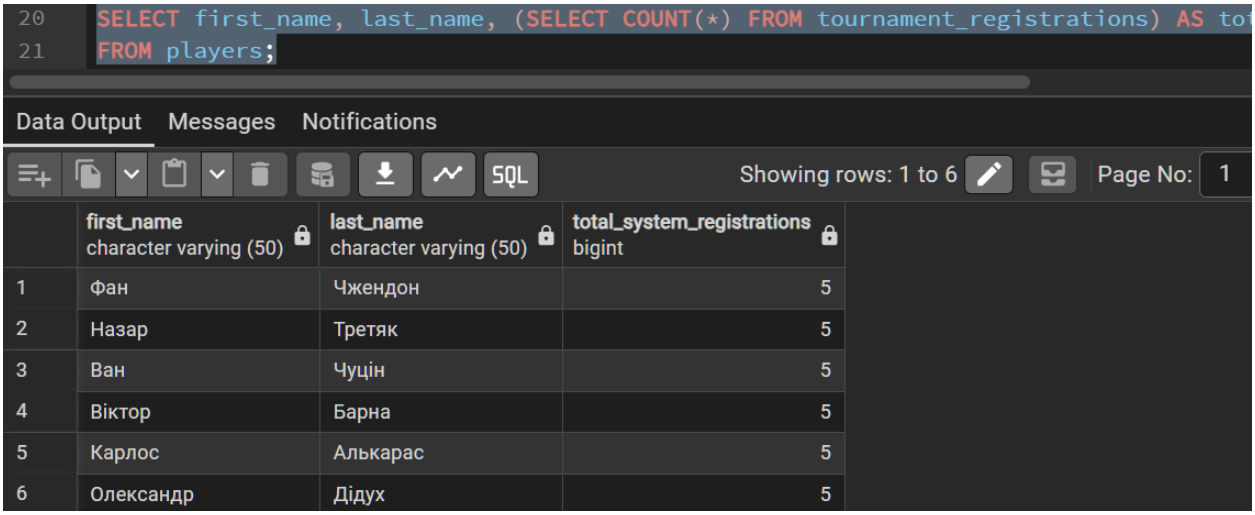


	name character varying (100)
1	Чемпіонат України

Тут шукає назви турнірів, у яких проводився етап "Фінал". Спочатку підзапит знаходить ID таких турнірів у таблиці матчів, а потім основний запит виводить їхні назви за допомогою оператора IN.

```
SELECT first_name, last_name, (SELECT COUNT(*) FROM
tournament_registrations) AS total_system_registrations

FROM players;
```



	first_name character varying (50)	last_name character varying (50)	total_system_registrations bigint
1	Фан	Чжендон	5
2	Назар	Третяк	5
3	Ван	Чуцін	5
4	Віктор	Барна	5
5	Карлос	Алькарас	5
6	Олександр	Дідух	5

Виводяться імена та прізвища всіх гравців, а поруч у додатковому стовпці показує загальну кількість реєстрацій на турніри в усій системі, за допомогою підзапиту прямо в блоці SELECT.

Висновок: під час виконання цієї лабораторної роботи я здобула практичні навички роботи з аналітичними SQL-запитами (OLAP). Всі запити виконалися без помилок у середовищі PostgreSQL та видали очікувані результати, що підтверджує успішне досягнення всіх цілей лабораторної роботи.